

COL 380 : Assignment 0

Vikash Kumar Meena : 2021CS10120

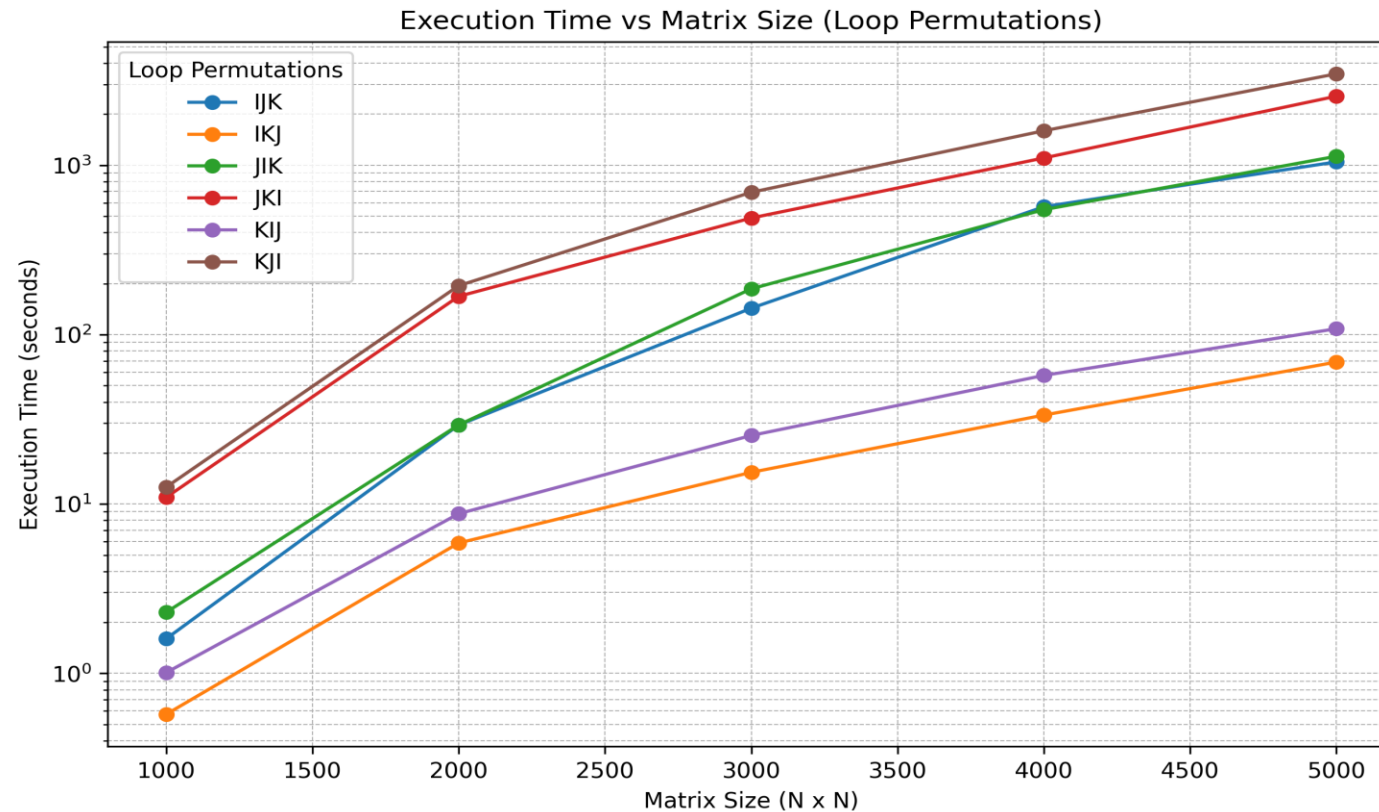
January 2025

The goal of the assignment is to implement and analyze six loop permutations for sequential matrix multiplication, measure their performance on varying matrix sizes (1000x1000 to 5000x5000), and profile them using **gprof** and **perf**. The focus is on understanding execution time, cache hit rates, cache misses, and CPU time distribution across functions (matrixMultiply, readMatrix and writeMatrix). The assignment requires optimizing the code with **-O3** compiler flags, comparing results from both tools, and identifying the best permutation in terms of execution time and cache efficiency. The findings are to be presented with graphs and raw data to explore how loop ordering impacts computational performance.

- Experiment 1

Execution Time vs Matrix Size (Average of Three Runs)

In this experiment, we plotted the average execution time over 3 runs for each matrix size and loop permutation. The time reported by our Python script was used as the Y-axis, while matrix size was plotted on the X-axis. This allowed us to observe how loop orders scale with increasing matrix sizes.

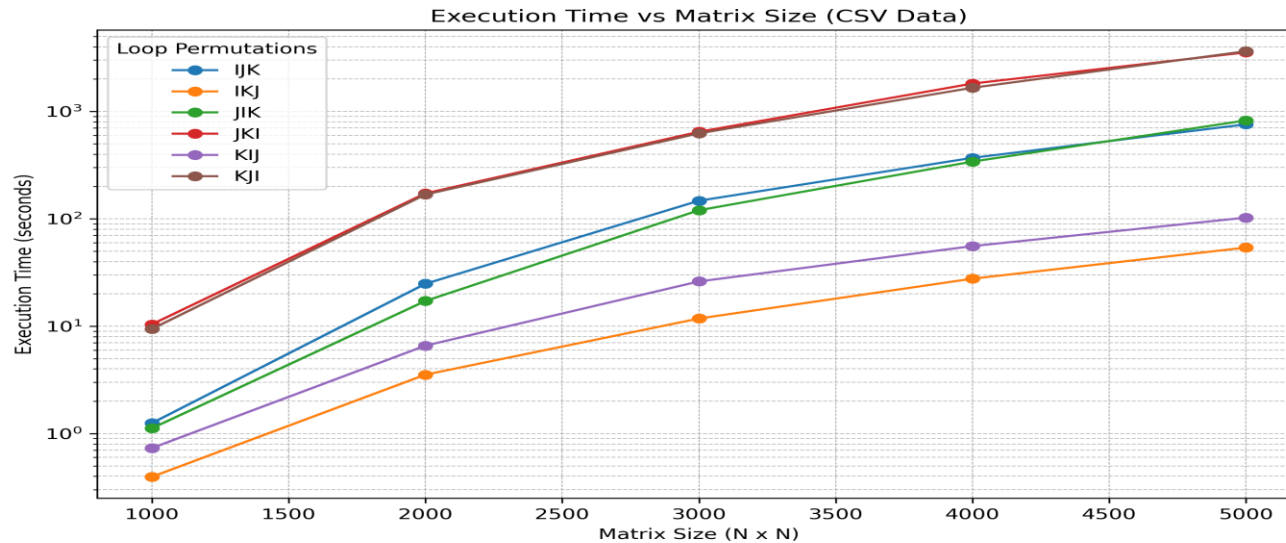


Comments

- Execution times grows with increasing matrix size, as expected for a cubic-complexity operation.
- Loop permutation IKJ and KIJ consistently outperformed others due to their optimized memory access patterns and more instruction per cycle value.
- Variability between loop orders was more pronounced for larger matrix sizes, highlighting the importance of efficient memory utilization at scale.

Experiment 2

This experiment replicated the first but used total user times as reported by **perf** for the Y-axis. The comparison highlights discrepancies between our Python script's timing and **perf** measurements.

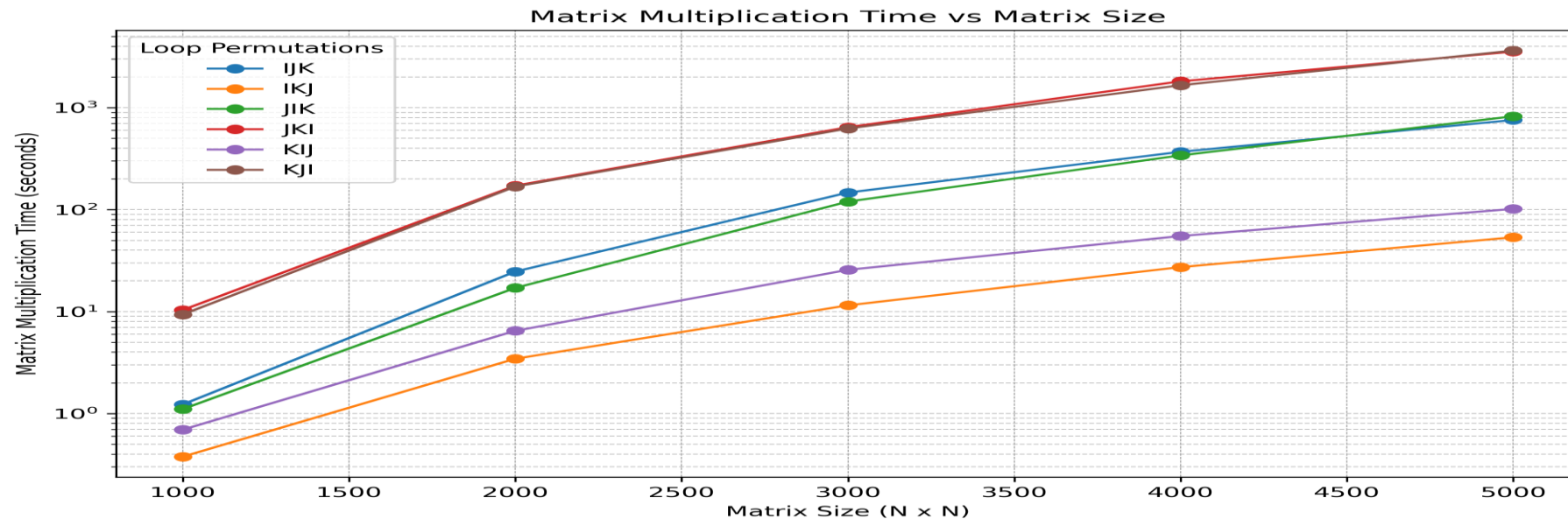


Comments

- With increase in matrix size user time increases non linearly, same as experiment 1.
- Permutations IKJ and KIJ have the lowest user time, while JKI and KJI have highest user time same as experiment 1.

Experiment 3

In this experiment, we measured the execution time strictly for the matrix multiplication computation, excluding file I/O and memory allocation times and time taken by readMatrix and writeMatrix methods. This time was extracted from the performance logs and plotted against matrix sizes.

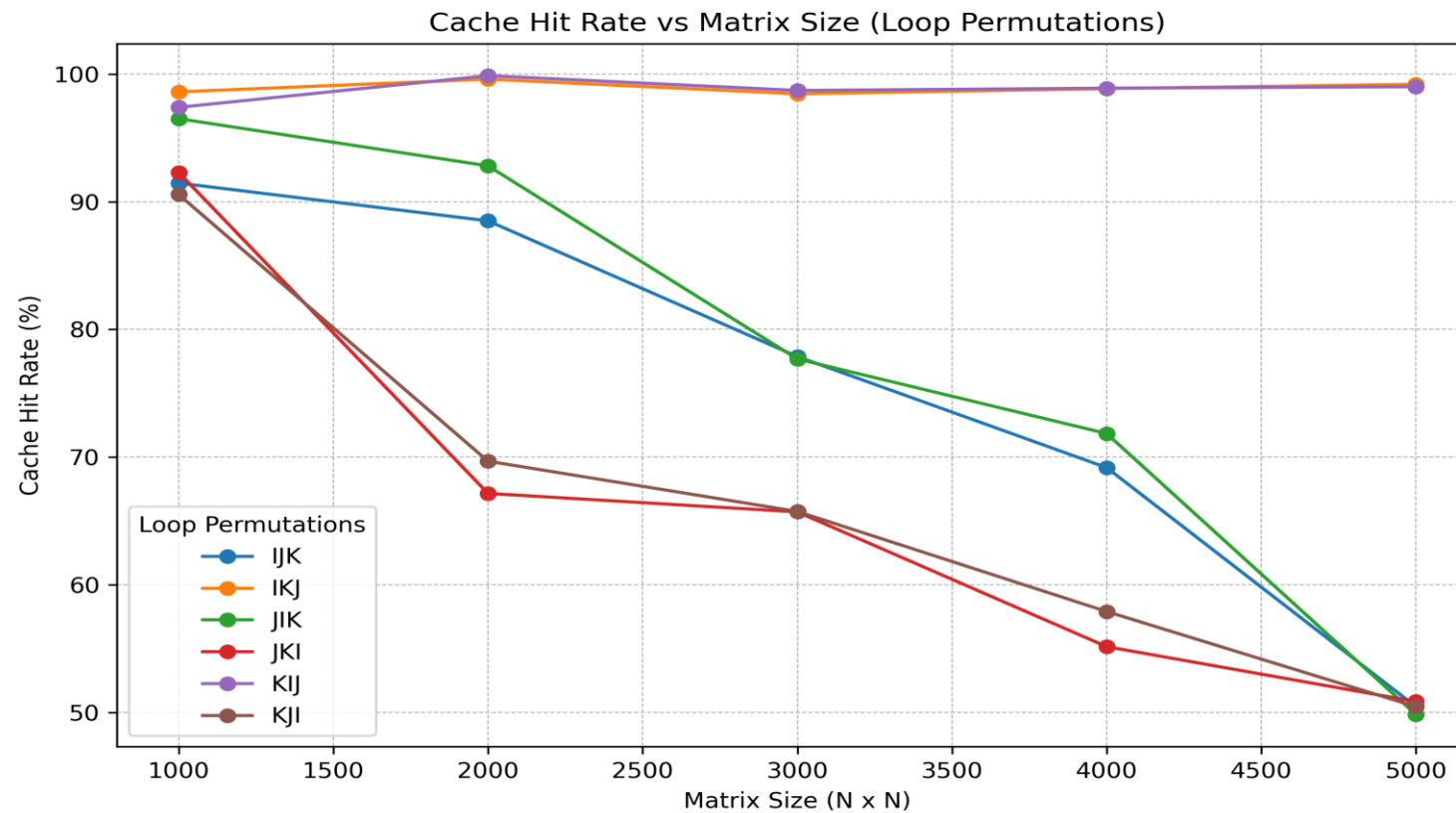


Comments

- With increase in matrix size user time in matrix Multiplication increases non linearly.
- Permutations IKJ and KIJ have the lowest user time, while JKI and KJI have highest user time in matrix multiplication.

Experiment 4

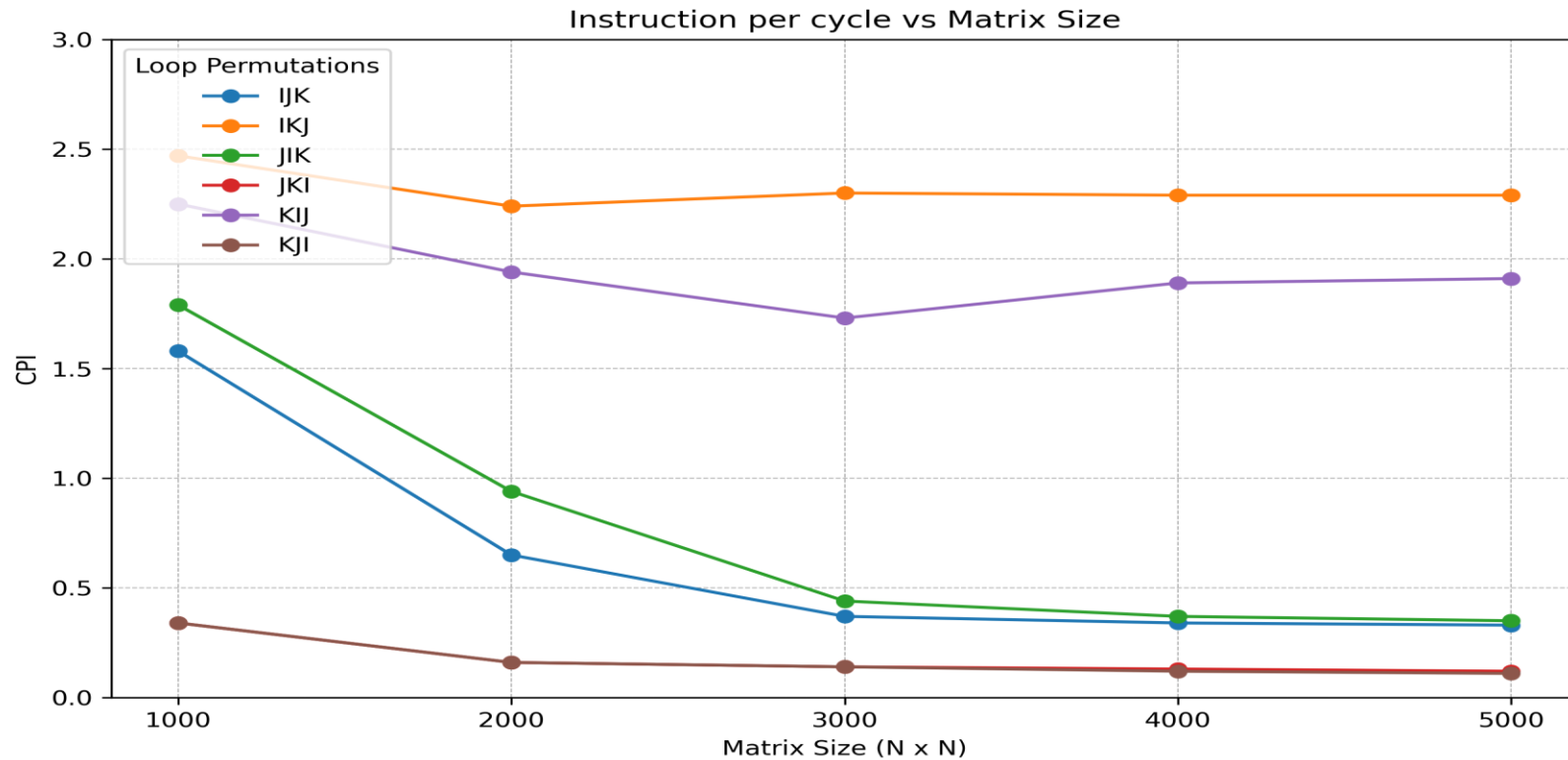
This experiment evaluated the cache efficiency of each loop permutation by plotting the cache-hit rate (calculated from perf **logs**) against matrix size.



Comments

- The gaps between permutations increases with matrix size.
- Permutation IKJ and KIJ have the lowest cache hit rate.
- Permutation JKI and KJI have the best cache hit rate.

This is the graph for instruction per cycle and matrix size for each permutation of matrix.



- Permutation IKJ and KIJ have instructions per cycle as compared to others.

Inferences

Cache Hit Rate vs Matrix Size (Loop Permutations) :

Trends Observed

- The cache hit rate decreases as the matrix size increases for all loop permutations.
- Smaller matrix sizes have a higher cache hit rate because the data fits better in the cache, leading to efficient memory usage.
- For larger matrix sizes, the hit rate drops as more data needs to be fetched from main memory.

Permutation Comparisons:

- IKJ and KIJ maintain a consistently high cache hit rate across all matrix sizes, indicating better memory locality for these permutations.
- JIK, JKI, and KJI exhibit a more significant decline in cache hit rate as the matrix size increases.
- Permutation IJK shows a moderate decline in cache hit rate, sitting between the best and worst-performing permutations.

Key Observation:

- Loop orders that access data in a manner aligned with the memory layout (like IKJ and KIJ) perform better in terms of cache utilization.

Instructions Per Cycle (CPI) vs Matrix Size:

Trend Observed:

- CPI decreases as the matrix size increases for most permutations, indicating less computational efficiency as matrix size grows.
- This is likely due to the increased memory latency caused by a reduced cache hit rate.

Permutation Comparisons:

- IKJ and KIJ again perform the best, maintaining a lower and more stable CPI, signifying higher computational efficiency.
- IJK and JIK show an intermediate level of efficiency with moderate CPI values.
- JKI and KJI have the highest CPI values, indicating they are the least efficient permutations for matrix multiplication.

Key Observation:

- Lower CPI correlates with higher cache hit rates, as better memory locality reduces the penalty of accessing data from slower memory.

General Insights:

Memory Locality Matters:

- The choice of loop permutation significantly impacts cache efficiency and, in turn, computational efficiency (CPI). Access patterns like IKJ and KIJ align better with memory hierarchy, minimizing cache misses.

Impact of Matrix Size:

- Smaller matrix sizes benefit from higher cache hit rates, as more of the data can fit into faster memory levels like L1 and L2 caches.
- Larger matrix sizes overflow cache capacities, leading to increased memory latency and reduced efficiency.

Optimization Takeaway:

- Loop permutations should be chosen carefully, especially for large matrix sizes, to optimize performance. IKJ and KIJ are preferable based on both cache hit rate and CPI metrics.

Experiment 5

In this experiment the performance of different loop permutations in terms of cache hit rate and execution time across various matrix sizes. The objective is to identify the permutation with the best average cache hit rate and the one with the minimum execution time.

Observed Result

- Best Cache Hit Rate (Averaged) : It was observed for permutation **IKJ** with cache hit rate of 98.95%.
- Best Execution Time : It was observed for permutation **IKJ** which is 19.474 seconds.

The best permutation is permutation **IKJ**. The best permutation does not depend on the size because:

- Permutations 1(IKJ) and 4(KIJ) have the highest cache hit rate, with 1 having higher among the two, and it remains throughout the graph, across all matrix sizes. The best permutation in terms of high cache hit rate does not depend on the matrix size.
- Permutations 1(IKJ) and 4(KIJ) have the lowest execution time, with the gap with other permutations being highest for matrix size 5000x5000. Permutation 1 is the best in terms of execution time. In terms of execution time, We see that permutation 1 and 4 perform almost similarly, regardless of matrix size, so it does not depend on matrix size. Without using perf also, permutation 1 is the best performing in terms of execution time, so it does not depend on matrix size.
- Permutations 1(IKJ) and 4(KIJ) have high instruction per cycle. And permutation 1(IKJ) outperforms
- permutation 4(KIJ) along with other permutations regardless of matrix size.
- Thus permutation 1(IKJ) outperforms all of the other permutations in terms of **high instruction per cycle, low execution time and high cache rate** regardless of matrix size hence it is the best permutation.

Experiment 6 : In this experiment, The percentage of time spent in each function (e.g. readMatrix, writeMatrix, matrixMultiplyXYZ) was analyzed using both **perf** and **gprof** tools. The results were compared to assess consistency between the two tools.

Date Using Perf Tool

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	97.93	0.3	0.0
2000	99.33	0.1	0.0
3000	99.74	0.04	0.0
4000	99.82	0.03	0.0
5000	99.95	0.02	0.0

Table 1: Permutation 0 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	95.16	0.4	0.00
2000	97.35	0.23	0.0
3000	97.58	0.36	0.0
4000	98.54	0.25	0.02
5000	99.1	0.2	0.00

Table 2: Permutation 1 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	98.41	0.0	0.0
2000	99.3	0.12	0.0
3000	99.71	0.07	0.0
4000	99.82	0.03	0.0
5000	99.86	0.02	0.0

Table 3: Permutation 2 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.43	0.19	0.0
2000	99.50	0.14	0.0
3000	99.59	0.1	0.0
4000	99.8	0.08	0.0
5000	99.93	0.06	0.0

Table 4: Permutation 3 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	94.59	3.4	1.2
2000	98.2	0.8	0.01
3000	98.25	0.7	0.0
4000	98.86	0.16	0.0
5000	99.1	0.08	0.0

Table 5: Permutation 4 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.25	0.07	0.0
2000	99.93	0.01	0.0
3000	99.94	0.03	0.0
4000	99.96	0.01	0.0
5000	99.98	0.002	0.0

Table 6: Permutation 5 Data

Data Using gprof Tool

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.40	0.10	0.00
2000	99.50	0.05	0.00
3000	99.52	0.03	0.00
4000	99.54	0.02	0.00
5000	99.57	0.012	0.00

Table 13: Permutation 0 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.10	0.30	0.00
2000	99.40	0.10	0.00
3000	99.43	0.09	0.00
4000	99.49	0.06	0.00
5000	99.56	0.03	0.00

Table 14: Permutation 1 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.40	0.10	0.00
2000	99.50	0.10	0.00
3000	99.53	0.09	0.00
4000	99.55	0.07	0.00
5000	99.58	0.02	0.00

Table 15: Permutation 2 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.40	0.10	0.00
2000	99.50	0.07	0.00
3000	99.52	0.05	0.00
4000	99.54	0.04	0.00
5000	99.58	0.01	0.00

Table 16: Permutation 3 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.00	0.40	0.00
2000	99.50	0.10	0.00
3000	99.53	0.08	0.00
4000	99.56	0.03	0.00
5000	99.58	0.01	0.00

Table 17: Permutation 4 Data

Row Dimension	% of User Time Taken by mm	% of User Time Taken by rm	% of User Time Taken by wm
1000	99.50	0.10	0.00
2000	99.52	0.10	0.00
3000	99.54	0.07	0.00
4000	99.57	0.04	0.00
5000	99.59	0.01	0.00

Table 18: Permutation 5 Data

Analysis of Performance Data from gprof and perf

The data provides insights into the performance characteristics of different permutations (0 to 5) when executing matrix operations, specifically focusing on the percentage of time spent in three function (readMatrix, writeMatrix and matrixMultiplyXYZ).

gprof Analysis:

- Matrix Multiplication : Dominates execution time (>99% for all permutations). **Permutation 5** took the highest time in matrix multiplication, peaking at 99.99% for 5000x5000 matrices.
- Read Matrix : Contributes minimally (<0.4%), decreasing with matrix size.
- Write Matrix : Negligible across all permutations.

Perf Analysis:

- Matrix Multiplication : Major contributor, though slightly lower than **gprof** (e.g., 97.93% for 1000x1000). **Permutation 5** took the highest time matrix multiplication across all sizes.
- Read Matrix : More noticeable for smaller matrices (e.g., 3.4% for Permutation 4 at 1000x1000).
- Write Matrix : Minimal but greater than what observed via **gprof**, with rare non-zero values (e.g., 1.2% for Permutation 4 at 1000x1000).

Combined Insights:

Permutation 5 consistently take the larger time across all matrix size, especially for larger matrices.

Larger Matrices: Both tools agree that matrix size reduces the impact of **readMatrix** and **writeMatrix** with dominating matrixMultiplyXYZ execution time.